



Test, Build and Deploy

Persistent Interactive | Persistent University



Key Learning Points

- What is Unit Testing
- Jasmine framework
- How to run test cases in Angular application
- Understand a spec file
 - TestBed
 - Fixture
 - Component instance
- Create an Angular bundle
- Deploy on Local server

Overview

- Unit testing involves testing small units of an application like classes and functions.
- **Jasmine** is a JavaScript testing framework that enables to test angular applications.
- It follows the **BDD (Behavior Driven Development)** style of writing test cases.
- BDD approach uses a descriptive style to define test-cases which focuses on behavior of a piece code and is more readable even to a non-technical person.

Sample Test case in Jasmine

```
describe('Hello world', () => {  
  it('says hello', () => {  
    expect(helloWorld())  
      .toEqual('Hello world!');  
  });  
});
```

- **describe()**: Defines a TestSuite i.e a collection of test cases.
- **it()**: Defines a individual Test case
- **expect()**: Used to define an expectation of the output.
- **toEqual()**: Matcher for the expect.

Equality and patterns

- `expect(mixed).toEqual(mixed);`
- `expect(mixed).toMatch(pattern);`
- `expect(array).toContain(member);`
- `expect(instance).toBe(instance);`
- `expect(mixed).toBeDefined();`
- `expect(mixed).toBeFalsy();`
- `expect(mixed).toBeTruthy();`
- `expect(mixed).toBeNull();`
- <https://jasmine.github.io/2.0/introduction.html>

Karma

- Karma is a command line tool which lets us spawn browsers and run jasmine tests.
- It can also watch source files for changes and re-run the tests automatically.
- Jasmine and Karma together are a popular combination while unit testing Angular code.
- When creating Angular projects using the Angular CLI it defaults to creating and running unit tests using Jasmine and Karma.
- All the related modules are also specified in the `package.json` and loaded on project creation.
- The default structure of a new Angular project already contains spec files.
- The spec files have the basic structure of unit test cases.

app.component.spec.ts

- Checking if component instance is created.

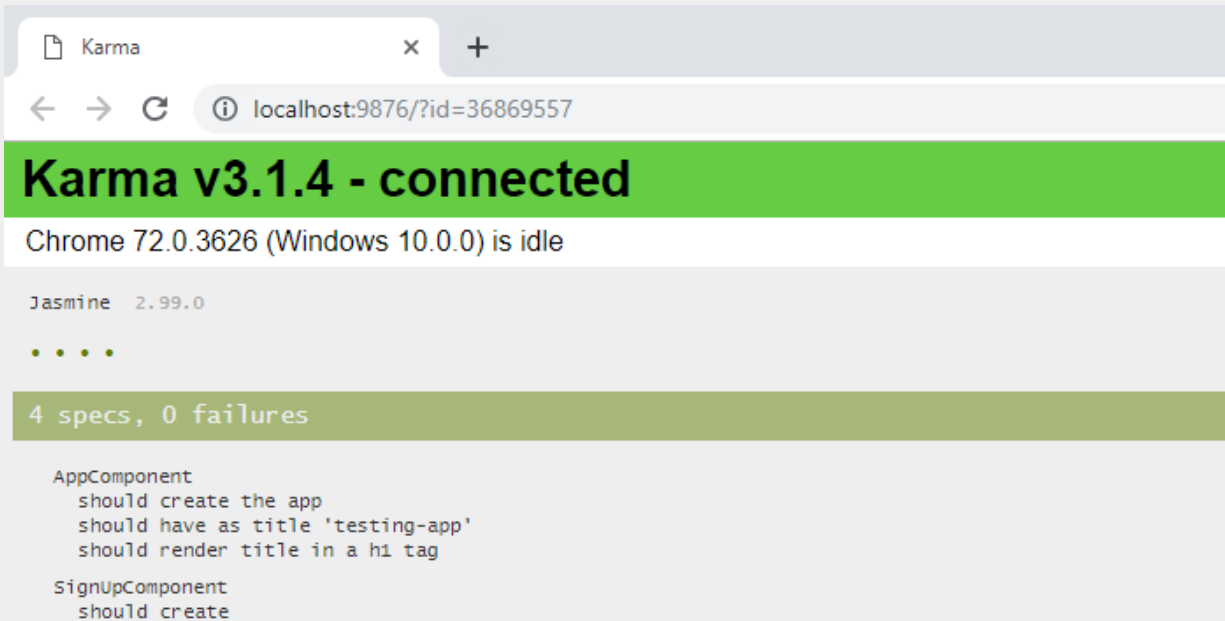
```
it('should create the app', () => {  
  const fixture =  
    TestBed.createComponent(AppComponent);  
  const app = fixture.debugElement.componentInstance;  
  expect(app).toBeTruthy(); });
```

Running tests in Angular project

- Run the below command to run all test cases for the project.

ng test

- Browser is opened up and results are displayed.



Test Bed

- We configure a testing module using the TestBed class.

```
beforeEach(() => {  
  TestBed.configureTestingModule({  
    declarations: [LoginComponent],  
    providers: [AuthService]  
  });  
});
```

- This creates a test *Angular Module* .
- We configure it in the same way as we would configure a normal NgModule.

Fixtures

- A fixture is a wrapper for a component and its template.

```
fixture = TestBed.createComponent(LoginComponent);
```

- Fetch a component from fixture.

```
component = fixture.componentInstance;
```

- Fetch the injected service.

```
authService = TestBed.get(AuthService);
```

Build the Angular Project

- Build the Project with the below command

`ng build`

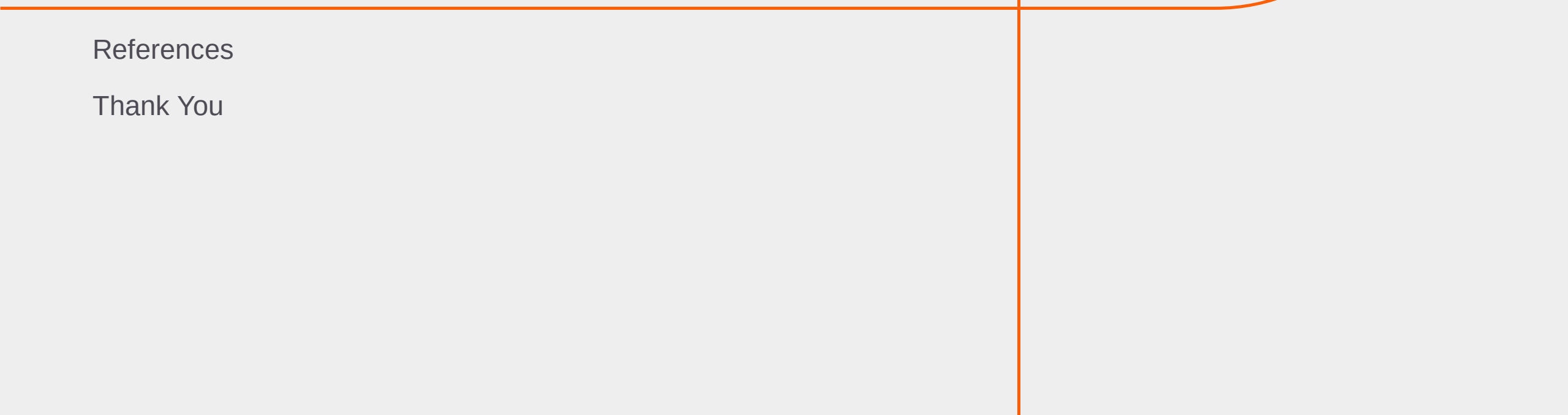
- This creates a bundle in the folder 'dist' with the minified application files.
- The dist folder contains the 'index.html' page which in turn loads the main.js file that loads the Angular Root module and components.

Summary: Session

With this we have come to an end of our session, where we discussed about

- How to write unit test cases in Angular
- How to test components
- How to run spec files using Angular CLI
- How to build and deploy an Angular application

Appendix



References

Thank You

References

- <https://medium.com/swlh/angular-unit-testing-jasmine-karma-step-by-step-e3376d110ab4>
- <https://medium.com/swlh/angular-unit-testing-jasmine-karma-step-by-step-e3376d110ab4>
- <https://angular.io/cli/build>
- <https://angular.io/guide/deployment>

Key Contacts :

Persistent University :

- Archana Ghatigar
archana_ghatigar@persistent.co.in



Thank you!

Persistent Interactive | Persistent University

